

Läsa från en fil

Att läsa från en fil är enkelt i C++. Exakt samma syntax används som då man läser från `cin`. Först måste man dock öppna en fil för läsning. Detta sker genom att skapa en instans av klassen `ifstream`. Denna tar som argument ett filnamn som skall vara en C-sträng (inte `string`). Allmänt görs det på följande sätt:

```
#include <fstream>

ofstream namn (filnamn);
```

Alla filrelaterade streamklasser finns definierade i headerfilen `fstream`. Om man vill initiera en `ifstream` med ett filnamn som finns i en `string` man man använda dess metod `c_str()` för att få en C-sträng som kan användas. En alternativ konstruktor finns som tar som andra argument en *bitmask* som representerar önskad funktionalitet hos filen:

```
ofstream namn (filnamn, openmode mask);
```

De olika konstanter som kan or:as ihop är definierade i klassen `ios`. De värden som påverkar `ifstream` är följande:

- `in` öppnar filen för läsning. Detta är standardvärdet ifall man skapar en `ifstream` med endast en parameter.
- `binary` specificerar att data skall läsas i binär form istället för text.

Man bör alltid kolla ifall filen kunde öppnas innan man försöker läsa data från den. Ifall filen inte existerar eller programmet inte har rätt att läsa filen kommer objektet inte att kunna användas för läsning. Man kan kolla detta med metoden `good()` eller med den överlagrade operatorm `!` enligt följande:

```
ifstream Fil ( "filnamn");

// använd antingen:
if ( ! Fil ) {
    // kunde inte öppna filen
}

// eller:
if ( ! Fil.good () ) {
    // kunde inte öppna filen
}
```

Vilken man använder har ingen betydelse.

Vi kan nu skapa ett enkelt program som öppnar en fil för läsning och läser samt räknar alla ord som finns i filen:

```
#include <fstream>
#include <string>
```

```
int main (int argc, char * argv[]) {
    int Antal = 0;
    string Ord;

    // har vi tillräckligt parametrar?
    if ( argc != 2 ) {
        cout << "Fel antal parametrar!" << endl;
        cout << "Användning: " << argv[0] << " filnamn" << endl;
        exit ( EXIT_FAILURE );
    }

    // öppna en fil
    ifstream Data ( argv[1] );
    if ( ! Data ) {
        // kunde inte öppna filen
        cout << "Kunde inte öppna filen: " << argv[1] << endl;
        exit ( EXIT_FAILURE );
    }

    // iterera över all ord i filen
    while ( Data >> Ord ) {
        // öka antalet lästa ord
        Antal++;
    }

    // skriv ett svar
    cout << "Läste " << Antal << " ord." << endl;
}
```

Notera att vi inkluderar headerfilen `fstream`, och inte `iostream` som normalt. Data kan läsas från en `ifstream` exakt på samma sätt som från `cin`, d.v.s. alla primitiva datatyper kan läsas så väl som strängar.

Avsluta inläsning

Inläsning avslutas vanligen då man läst in den mängd data som behövs eller då slutet på filen nåtts. Den första möjligheten är lätt att hantera, eftersom programmet då vet hur den inlästa filen ser ut och vad som kan förväntas. Om programmet däremot inte vet någonting om den lästa filen vill man kanske läsa till till filens slut, d.v.s. *end-of-file*. Man kan kolla om en `ifstream` (och även en `istream` som t.ex. `cin`) har nått slutet på filen med metoden `eof()`. Denna returnerar `true` då slutet nåtts och `false` ifall mera data kan läsas. Programmet ovan använder sig av funktionaliteten att alla stream-klasser alltid returnerar sig själv vid läsning och skrivning med `>>` respektive `<<`, och en misslyckad läsning returnerar 0.

```
// iterera över all ord i filen
while ( Data >> Ord ) {
    // öka antalet lästa ord
    Antal++;
}
```

Så stream:en `Data` returneras efter varje `>>`-operation, och den sista operationen som misslyckas returnerar 0 eller `false`. För mera information om den överlagrade operatören fungerar se [Kapitel 22](#) och [Kapitel 21](#). Vi kunde skriva om den del av koden som sköter inläsningen i programmet ovan till detta:

```
// iterera över all ord i filen
while ( ! Data.eof () ) {
```

```
// läs ett ord
if ( Data >> Ord ) {
    // öka antalet lästa ord
    Antal++;
}
}
```

Varför måste vi ha en `if`-sats runt inläsningen? Om vi inte har denna där kommer även den sista inläsningen som ju läser något som inte existerar, att även räknas med. Flaggan `eof()` sätts inte förrän *efter* att någon operation läst data som inte finns. Klassen kan inte veta ifall det finns mera data eller inte förrän någonting försöker läsa något som inte finns. På detta sätt räknar vi med endast de ord som vi lyckades läsa.

Ett alternativt sätt att åstadkomma samma funktionalitet är att använda `fail()` istället för `eof()` i koden ovan. Denna metod anger ifall nästa operation kommer att misslyckas eller inte. Den kan dock inte heller vet i förväg om nästa läsning kommer till filens slut, utan den kan endast på bas av status hos objektet i anropsögonblicket veta ifall nästa operation kommer att misslyckas.

Läsa teckenvis

Man kan förutom att läsa data från en `ifstream` (eller t.ex. `cin`) även läsa data tecken för tecken, eller ett antal tecken på en gång. Beroende på tillämpningen kan detta vara vad som behövs. För att läsa tecken för tecken används metoden `get()` med olika argument. Den enklaste formen används på följande sätt:

```
char Tecken
enFil.get ( Tecken );
```

Metoden `get()` läser alla tecken som finns i filen, även olika former av *whitespace* (mellanslag, tabulatorer m.m.), medan de olika formerna av `>>` hoppar över *whitespace*. man bör sålunda använda t.ex. `get()` då man vill åt *exakt* det som finns i en fil.

Andra versioner av `get()` tar som parametrar en C-sträng, d.v.s. en `char *`. Med dessa kan man läsa ett antal tecken upp till ett givet maximum eller tills ett givet termineringstecken påträffas. Dessa ser ut på följande sätt:

```
char Buffer [storlek];
enFil.get ( Buffer, storlek );
// eller
enFil.get ( Buffer, storlek, terminator );
```

Dessa metoder läser maximalt `storlek - 1` tecken in i den buffer som ges med som parameter. Denna måste förstås ha utrymme för alla dessa tecken. Det sista teckent sätts alltid till 0 för att terminera strängen. Så om storleken ges till 10 läses endast maximalt 9 tecken in medan det tionde sätts till `'\0'`. Metoden läser i den första utformningen ända tills ett radbytetecken (`'\n'`) hittas. I den andra formen kan man själv specificera vilket tecken som skall vara det terminerande. Vill man läsa rader är `'\n'` lämpligt, men vill man t.ex. läsa ord kan man separera på ett mellanslag istället med `' '`.

Dessa två varianter av `get()` har ett problem. De läser inte bort det tecken som terminerade inläsningen. Om vi t.ex. terminerar på ett `'\n'` läser `get()` inte bort teckent, utan det finns kvar då nästa läsning utförs. Vi måste manuellt läsa bort tecknet, t.ex. med en annan `get()`. En aning arbetsdrygt. Vi kan göra ett program som räknar antalet rader i ett program (en form av programmet `wc`) enligt följande:

```
#include <fstream>

int main (int argc, char * argv[]) {
    int Antal = 0;
    char Buffer [250];

    // kontrollera argument till main()
    ...

    // öppna en fil, samma
    ifstream Data ( argv[1] );

    // felhantering från exemplen ovan
    ...

    // läs rad för rad
    while ( Data.get ( Buffer, 250 ) ) {
        // läs det '\n' som finns
        Data.get (Newline);

        // öka antalet lästa tecken
        Antal++;
    }

    // skriv ett svar
    cout << "Läste " << Antal << " tecken." << endl;
}
```

Vissa delar är bortlämnade, men de är identiska med programmen ovan. Notera att vi har en extra

```
Data.get (Newline);
```

för att läsa bort det radbytetecken som terminerade inläsningen. Programmet antar att de inlästa raderna är under 250 tecken långa, annars utförs flera läsningar på samma rad och raden räknas multipelt med i antalet rader och vi får ett felaktigt resultat.

En sista form av att läsa teckenvis är med metoden `read()`. Den skiljer sig från `get()` i och med att den inte har något speciellt tecken som terminerar inläsningen, utan det givna antalet tecken läses alltid in. Den terminerar dock då slutet på filen nåts. Metoden placerar inget `'\0'` som det sista tecknet, utan den läser hela den givna buffern full med data (om så mycket data kunde läsas). Använd denna endast om du vet vad du gör! Kan användas för binär läsning ([avsnittet *Läsa binär data*](#)).

Läsa radvis

Ett alternativ till att läsa filer teckenvis är att läsa radvis. Detta utförs med metoden `getline()` som även den finns i flera olika varianter. De två normala varianterna fungerar exakt såsom de olika varianterna av metoden `get()` (se [avsnittet *Läsa teckenvis*](#)), med den signifikanta skillnaden att `getline()` även läser bort det terminerande tecknet. Vi kan således skriva om huvudslinga för vårt ordräkningsprogram på följande sätt:

```
char Buffer [250];
....

// läs rad för rad
while ( Data.getline ( Buffer, 250 ) ) {
    // öka antalet lästa rader
    Antal++;
}
```

```
}
```

Vi måste inte läsa bort ett `'\n'` med en extra `get()`, utan den läses av `getline()`. Vi kan även använda den andra formen av metoden `getline()` för specificera ett annat termineringstecken än `'\n'`.

För klassen `string` finns en extra *funktion* definierad som heter `getline()`. Notera ettd et inte är frågan om en metod! Denna funktion fungerar på samma sätt som `getline()` ovan, men man måste explicit ge med en `ifstream` eller `istream` som första parameter. Strängen ges som andra parameter. Vi behöver inte ge någon storlek för denna funktion, eftersom `string` växer vid behov. Detta är det mest robusta sättet att läsa en rad, men även det långsammaste. För det mesta lönar det sig dock att offra en aning hastighet och vinna robusthet. Vi kan skriva om slingan ovan till:

```
string Buffer;
...

// läs rad för rad
while ( getline ( Data, Buffer ) ) {
    // while ( Data.getline ( Buffer ) ) {
    // öka antalet lästa rader
    Antal++;
}
```

Notera att vi använder en funktion och inte en metod. Funktionen `getline()` är definierad i filen `<string>`.

Ignorera tecken

Ibland kanske man vill kunna läsa tecken ur en fil eller annan in-stream och ignorera dem tills något specifikt tecken läses. Detta kan göras med metoden `ignore()` som tar som parametrar antalet tecken som maximalt skall ignoreras och ett termineringstecken som avslutar ignoreringen. Kan användas t.ex. på följande sätt för att ignorera max 80 tecken eller tills nästa `;` läses:

```
enFil.ignore (80, ';');
```

De tecken som läses sparas ingenstans. Båda parametrarna har standardvärden som gör att `ignore()` utan parametrar ignorerar ett tecken och avslutar då filens slut nåts. Vi kan skriva om programmet ovan som läser rader m.h.a. `get()` (se [avsnittet *Läsa teckenvis*](#)) till följande:

```
// läs rad för rad
while ( Data.get ( Buffer, 250 ) ) {
    // ignorera det '\n' som finns
    Data.ignore ();

    // öka antalet lästa rader
    Antal++;
}
```

Slingan blir en aning lättare att förstå.

Läsa binär data

Man kan även med `ifstream` läsa data som är i *binär* form, d.v.s. inte ASCII. Filer i binär form kan sällan läsas av andra program än det program som skapat filen, d.v.s. man kan inte titta meningsfullt

på filens innehåll med t.ex. `more` eller `notepad`, utan filen verkar innehålla en massa skräp. Om vi t.ex. skriver ut talet 123456 (en `int`) till en textuell fil skrivs bokstäverna '1', '2', '3', '4', '5' och '6' till filen. Skriver vi däremot i binär form skrivs de fyra (eller annat antal) *bytes* ut som datatypen består av.

För att öppna en fil för läsning i binärt format använder man sig av flaggan `ios::binary` då man öppnar filen. T.ex. kan vi öppna en binär fil för läsning på följande sätt:

```
ifstream BinData ( "register.dat", ios::binary );
```

Notera att denna flagga inte har någon verkan under Unix! För att sedan läsa binär data från filen *borde* man kunna använda normala `>>`, men under Unix bör man använda sig av metoden `read()`. Ett program som läser in en serie `int`:s från en fil kan skrivas på följande sätt:

```
#include <fstream>
#include <stdlib.h>

int main () {
    int Tmp;

    // öppna filen
    ifstream In ( "Test.bin");

    // kunde filen öppnas?
    if ( ! In ) {
        // kunde inte öppna filen!
        cout << "Kunde inte öppna filen 'Test.bin'!" << endl;
        exit ( EXIT_FAILURE );
    }

    // läs in tal från filen
    while ( In.read ( &Tmp, sizeof(int) ) ) {
        cout << "Läste: " << Tmp << endl;
    }
}
```

Vi använder `sizeof(int)` för att portabelt kunna ta reda på storleken på en `int`, så att `read()` kan läsa korrekt antal bytes. Portabilitet är då man använder binära filer ganska sekundärt, eftersom man ändå i många fall inte kan utbyta binära datafiler mellan olika system p.g.a. olika storlekar på datatyper. Om man inte har ett absolut behov av binär data bör man använda sig av datafiler i textform. Undantag är bl.a. fall där filer i textform blir för stora, eller där bandbredden är begränsad (t.ex. nätverk).

[Förutgående](#)
Filhantering

[Hem](#)
[Upp](#)

[Nästa](#)
Skriva data till fil